CSCI 480 / CIS 680 – Advanced Databases

Research Lab Project: Investigating Index Selectivity and Query Optimization

Database System: MySQL

Environment: GitHub Codespaces

Project Type: Guided Experimental Investigation

Total Points: 100

• Project Overview

Database indexes can improve query performance, but using an index is not always faster than scanning a table. Database management systems use query optimizers to estimate the costs of different execution strategies.

In this project, you will conduct experiments to investigate how index selectivity affects query execution plans and performance.

You will be provided with SQL statements to construct the experimental database. However, you must independently conduct the experiments, collect measurements, interpret the execution plans, and draw conclusions supported by evidence.

This project has the same requirements for all CSCI 480 and CIS 680 students.

• Learning Objectives

Upon completing this project, you should be able to:

• Explain the relationship between index selectivity and query performance.

• Use `EXPLAIN ANALYZE` to investigate MySQL query execution plans.

• Compare index-based access with full table scans.

• Design and conduct a controlled database performance experiment.

• Interpret experimental results and connect them to database optimization concepts.

• Communicate research findings and limitations.

Part 1. Create the Experimental Database (10 points)

Open your MySQL environment in GitHub Codespaces and execute the following SQL statements.

Step 1: Create the database and table.

```
CREATE DATABASE IF NOT EXISTS index_research;
USE index_research;

CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100) NOT NULL,
    adm_year INT NOT NULL,
    gpa DECIMAL(3,2) NOT NULL
) ENGINE=InnoDB;
```

Step 2: Create a helper table.

```
CREATE TABLE Digits (
    n INT PRIMARY KEY
);

INSERT INTO Digits (n)
VALUES (0),(1),(2),(3),(4),
       (5),(6),(7),(8),(9);
```

Step 3: Generate 500,000 student records.

```
INSERT INTO Students
    (student_id, student_name, adm_year, gpa)
SELECT
    x.id,
    CONCAT('Student_', x.id),
    CASE
        WHEN x.id % 100 = 0 THEN 2020
        WHEN x.id % 10 = 0 THEN 2021
        ELSE 2022
    END,
    2.00 + (x.id % 201) / 100.0
FROM (
    SELECT
        a.n + 10*b.n + 100*c.n
        + 1000*d.n + 10000*e.n
        + 100000*f.n + 1 AS id
    FROM Digits a
    CROSS JOIN Digits b
    CROSS JOIN Digits c
    CROSS JOIN Digits d
    CROSS JOIN Digits e
    CROSS JOIN Digits f
) AS x
WHERE x.id <= 500000;
```

This operation may take some time depending on your Codespaces environment.

Step 4: Verify the data.

```
SELECT COUNT(*) AS total_students
FROM Students;

SELECT adm_year, COUNT(*) AS student_count
FROM Students
GROUP BY adm_year
ORDER BY adm_year;
```

Your tasks:

• Record the total number of records.

• Record the number and percentage of students for each admission year.

• Explain why this distribution is useful for studying index selectivity.

Part 2. Examine Query Performance Without a Secondary Index (15 points)

Initially, the `adm_year` column has no secondary index.

Execute the following statements:

```
EXPLAIN ANALYZE
```

```
SELECT *
FROM Students
WHERE adm_year = 2020;

EXPLAIN ANALYZE
SELECT *
FROM Students
WHERE adm_year = 2021;

EXPLAIN ANALYZE
SELECT *
FROM Students
WHERE adm_year = 2022;
```

For each query:

• Identify the access method reported by MySQL.

• Record the actual execution time.

• Record the actual number of rows returned.

• Explain whether MySQL scans the table or uses an index.

Run each query five times and use the median execution time for comparisons in subsequent parts.

Investigation question: Why might queries returning very different numbers of rows have similar execution times when no suitable secondary index exists?

Part 3. Create an Index and Repeat the Experiments (20 points)

Create a B-tree index on the admission-year column.

```
CREATE INDEX idx_students_adm_year
ON Students(adm_year);

ANALYZE TABLE Students;
```

Repeat the three queries from Part 2.

Run each query five times and record the execution times reported by `EXPLAIN ANALYZE`. Use the median of the five measurements for your comparison.

Complete the following table.

Admission Year	Percentage of Rows	Without Index (ms)	With Index Available (ms)	Access Method Chosen
2020				
2021				
2022				

Investigation questions:

• For which admission year does the index provide the greatest performance benefit?

• Does MySQL use the index for every query?

• Does creating an index always reduce query execution time?

• How does the percentage of matching rows influence the optimizer's decision?

Part 4. Compare Alternative Execution Strategies (20 points)

MySQL normally chooses an execution plan automatically. However, we can use index hints to investigate alternative strategies.

Experiment A: Encourage index-based access.

```
EXPLAIN ANALYZE
SELECT *
FROM Students FORCE INDEX (idx_students_adm_year)
WHERE adm_year = 2020;
```

Experiment B: Prevent use of the secondary index.

```
EXPLAIN ANALYZE
SELECT *
FROM Students IGNORE INDEX (idx_students_adm_year)
WHERE adm_year = 2020;
```

Repeat both experiments for admission years 2021 and 2022. Also run the corresponding queries without hints.

Verify the actual access method in each execution plan. Do not assume that the hint alone proves which method was used.

Complete the table using median execution times from five runs per condition.

Admission Year	Optimizer Choice (ms)	FORCE INDEX (ms)	IGNORE INDEX (ms)
2020			
2021			
2022			

Investigation questions:

• Which strategy is fastest for each admission year?

• Does MySQL's automatic optimizer choose the fastest strategy in your experiments?

• Can forcing index usage make a query slower? Explain using your results.

• What factors besides selectivity might affect query performance?

Part 5. Independent Research Experiment (20 points)

Design and conduct one additional experiment of your own.

You may choose one of the following directions or propose another related investigation.

Option A: Table Size

Investigate whether the benefits of indexing change as the number of records increases. Compare smaller and larger tables using the same query conditions.

Option B: Covering Index

Investigate whether an index becomes more useful when a query retrieves only columns contained in the index.

For example, compare:

```
EXPLAIN ANALYZE
```

```
SELECT *
FROM Students
WHERE adm_year = 2020;
```

with:

```
EXPLAIN ANALYZE
SELECT adm_year
FROM Students
WHERE adm_year = 2020;
```

Option C: Range Queries

Investigate whether index performance changes when using a range condition rather than an equality condition.

For example:

```
EXPLAIN ANALYZE
SELECT *
FROM Students
WHERE adm_year BETWEEN 2020 AND 2021;
```

For your independent experiment, you must:

• State a research question.

• Formulate a testable hypothesis before conducting the experiment.

• Describe your experimental design and the variables you will control.

• Execute the experiment and collect results.

• Explain whether the results support your hypothesis.

• Discuss at least one limitation of your experiment.

Part 6. Research Background and Final Report (15 points)

Find at least two relevant academic or technical sources discussing database indexing, query optimization, or execution-plan selection.

At least one source should be a research paper or scholarly publication. Use the sources to explain the motivation for your experiment and compare your findings with established database concepts.

Submit a concise report containing:

• Introduction: Research question and motivation.

• Background: Relevant concepts and related work.

• Methodology: Database configuration, queries, and experimental procedure.

• Results: Completed tables, execution-plan evidence, and at least one chart.

• Discussion: Interpretation of findings, unexpected behavior, and limitations.

• Conclusion: What did you learn about index selectivity and query optimization?

• References: Sources cited in the report.

Suggested length: 4–6 pages, excluding SQL code and detailed execution output.

Course Requirements

All students enrolled in CSCI 480 and CIS 680 must complete Parts 1–6, including one independent research experiment, a literature review using at least two relevant sources, and a final report.

The same workload, submission requirements, and 100-point grading rubric apply to all students, regardless of course level.

Submission Requirements

Submit through Canvas:

- A PDF report containing your experimental results and analysis.
- A `.sql` file containing the SQL statements used for the experiments.
- Evidence of representative execution plans, such as screenshots or copied text output.
- A brief presentation summarizing your research question, methodology, findings, and conclusions.

Grading Rubric

Component	Points
Database creation and verification	10
Baseline experiments	15
Indexed-query experiments	20
Execution-strategy comparison	20
Independent research experiment	20
Research background and final report	15
Total	100

Important: Your grade will be based on the correctness of your experimental procedure, the quality of your analysis, and your ability to support conclusions with evidence—not on whether indexing produces a particular performance improvement.

Unexpected or negative results are acceptable when properly investigated and explained.

Experimental Notes

- Use the same MySQL environment and table data when comparing access strategies.
- Avoid other resource-intensive operations during measurements.
- Repeat experiments because timing measurements may vary due to caching and system activity.
- Record the MySQL version and Codespaces configuration.
- Interpret both execution plans and execution times.
- A single execution time is not sufficient evidence for a general performance conclusion.
- Keep your SQL scripts so your experiments can be reproduced.